Annexe · fiches de référence

Séance du 09/11 : StateGraph, persistance, human-in-the-loop, sortie structurée, robustesse

Les fiches 🔎, les approfondissements et le code de chaque étape, dans l'ordre du cours · à garder ouverte



bstorm

L'étape 3 et le mini-défi

Fiches de référence, dans l'ordre des slides présentées

p. 3 Mini-défi : le code ajouté (1/2)

p. 4 Mini-défi : le code ajouté (2/2)

Mini-défi : le code ajouté (1/2)

le code ajouté de etape_03_stategraph.py à defi_03_escalade.py

```
# ▼ ligne 23
from typing import Literal                                          # ═══ NOUVEAU (défi 3)

# ▼ ligne 25 · remplace : from langchain_core.messages import HumanMessage, Syste
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage    # ═══ NOUVEAU (défi 3) : AIMessage

# ▼ ligne 127
# ═══ NOUVEAU (défi 3) : un nœud de plus, et la fonction qui choisit la route après le tri ═══
def escalader(state: EtatTicket) -> dict:
    return {"messages": [AIMessage(f"Ticket {state['priorite']} transmis à l'astreinte {state['file']} : "
                                   "un technicien vous rappelle dans les 15 minutes.")]}


def apres_tri(state: EtatTicket) -> Literal["escalader", "agent"]:
    return "escalader" if state["priorite"] == "P1" else "agent"



# ▼ ligne 142
builder.add_node("escalader", escalader)                  # ═══ NOUVEAU (défi 3)

# ▼ ligne 144 · remplace : builder.add_edge("trier", "agent")
builder.add_conditional_edges("trier", apres_tri)         # ═══ NOUVEAU (défi 3) : remplace add_edge("trier", "agent")
builder.add_edge("escalader", END)                        # ═══ NOUVEAU (défi 3)
```

🖇️ Mini-défi : le code ajouté (2/2)

le code ajouté de etape_03_stategraph.py à defi_03_escalade.py

```
# ▼ ligne 162 · remplace : dessiner(graphe, "etape_03_graphe")
dessiner(graphe, "defi_03_graphe")

# ▼ ligne 165 · remplace : with propagate_attributes(session_id="construction-etap (+1 ligne(s))
with propagate_attributes(session_id="construction-defi-3", user_id="agent-n1-042",
                          tags=["construction", "defi-3"]):

# ▼ ligne 179
            elif noeud == "escalader":                          # ═══ NOUVEAU (défi 3)
                print(f"   ▶ escalader : {maj['messages'][-1].content}")

# ▼ ligne 185 · remplace : print("🔭 Langfuse › Tracing › helpdesk (enfants : trier
print("🔭 Langfuse › Tracing › helpdesk (P1 : trier, escalader · sinon : trier, agent, tools)")
```

🧷 Persistance

Fiches de référence, dans l'ordre des slides présentées

p. 6 🔎 compile(checkpointer=...) : brancher la mémoire
p. 7 🔎 invoke(entrée, config) : adresser une conversation
p. 8 🔎 get_state : la photo du dernier checkpoint
p. 9 🔎 get_state_history : tous les checkpoints d'un thread
p. 10 🔎 SqliteSaver : des checkpoints dans un fichier
p. 11 Voyage dans le temps : rejouer et bifurquer
p. 12 Bifurquer : corriger un message et relancer
p. 13 🔎 update_state : écrire une branche
p. 14 Une purge qui s'applique vraiment
p. 15 🔎 delete_thread et list : effacer, inventorier
p. 16 Lire les messages d'une conversation
p. 17 🧰 Boîte à outils du mini-défi J2-01
p. 18 🧪 À vous : J2-01
p. 19 🧷 Étape 4 : le code ajouté (1/3)
p. 20 🧷 Étape 4 : le code ajouté (2/3)
p. 21 🧷 Étape 4 : le code ajouté (3/3)

🔎 compile(checkpointer=...) : brancher la mémoire

Le graphe ne change pas : on lui donne un endroit où écrire ses checkpoints

📦 langgraph 1.2 · langgraph-checkpoint 4.2

```python
from langgraph.checkpoint.memory import InMemorySaver

graphe = builder.compile(checkpointer=InMemorySaver())
# jalon J1 : construire_support(llm, checkpointer=…)
```

À quoi ça sert

compile() transforme le StateGraph en graphe exécutable. Avec un checkpointer, chaque super-step écrit un checkpoint (l'état complet + les nœuds à exécuter ensuite), rangé sous le thread_id du config. InMemorySaver garde ces checkpoints dans un dictionnaire du processus : idéal pour apprendre et tester, perdu au redémarrage du kernel.

↩️ Renvoie

Un CompiledStateGraph : invoke / ainvoke, stream, get_state, get_state_history, update_state. Un seul objet sert pour tous les threads.

Paramètre	Dans le cours	Pourquoi · autres choix
`checkpointer`	`InMemorySaver()`	défaut None : aucune mémoire · aussi SqliteSaver, PostgresSaver
serde (du saver)	défaut	sérialiseur : JsonPlusSerializer (messages, Pydantic…)
`interrupt_before / _after`	non utilisé	pause statique autour de nœuds nommés ; le cours préfère interrupt()
`store`	non utilisé	mémoire partagée ENTRE threads, pas la conversation
`name`	défaut ("LangGraph")	nom du graphe dans les traces

⚠️ Pièges

- Checkpointer sans thread_id dans le config : ValueError dès l'invoke
- Deux InMemorySaver différents = deux mémoires qui s'ignorent
- InMemorySaver en production : tout est perdu au redémarrage

🔎 invoke(entrée, config) : adresser une conversation

Le config porte le thread_id : même thread = même mémoire

📦 langgraph 1.2

```
config = {"configurable": {"thread_id": "conv-A"}}
graphe.invoke({"messages": [("user", "…")]}, config)
graphe.invoke(None, avant.config)     # reprendre / rejouer
```

À quoi ça sert

invoke exécute le graphe jusqu'au bout, ou jusqu'à une interruption, et rend l'état final. Le config est un dict (RunnableConfig) : sa clé configurable.thread_id dit au checkpointer quelle conversation charger avant l'exécution et où écrire ensuite. Nouveau thread_id = conversation vierge. ainvoke est la version async, obligatoire dès qu'un nœud a un timeout.

↩️ Renvoie

v1 : l'état final (dict), avec la clé __interrupt__ si le graphe s'est arrêté sur interrupt(). v2 : un GraphOutput, état dans .value, interruptions dans .interrupts.

Paramètre	Dans le cours	Pourquoi · autres choix
input	dict, None ou Command	dict : nouvelle entrée ; None : reprendre ; Command : répondre à interrupt()
thread_id	"conv-A"	une conversation, pas un utilisateur
checkpoint_id	via avant.config	repartir d'un checkpoint précis ; absent : le plus récent
version	"v2" (HITL)	défaut "v1" : un dict ; "v2" : GraphOutput (.value, .interrupts)
durability	défaut ("async")	"sync" : écrit avant l'étape suivante ; "exit" : à la fin
recursion_limit	défaut	clé du config : super-steps max (10007 en 1.2)

⚠️ Pièges

- Un thread_id réutilisé d'un exercice à l'autre : l'ancienne conversation revient
- invoke(None, config) sur un thread terminé : rien n'est exécuté

🔎 get_state : la photo du dernier checkpoint

Lire où en est une conversation, sans rien exécuter

📦 langgraph 1.2

```
etat = graphe.get_state(config_A)
etat.values["messages"]       # l'état du graphe
etat.next                     # () = terminé
```

À quoi ça sert

Méthode du graphe compilé : elle demande à son checkpointer le checkpoint le plus récent du thread désigné par le config (ou celui de checkpoint_id) et le présente sous forme de StateSnapshot. Rien n'est exécuté, aucun LLM n'est appelé. C'est l'outil de diagnostic de base : où en est la conversation, qu'attend le graphe, quelle interruption est en cours.

↩️ Renvoie

Un StateSnapshot (NamedTuple). Thread inconnu : values={} et next=(), sans erreur.

Paramètre	Dans le cours	Pourquoi · autres choix
config	config_A	le thread_id ; ou un checkpoint_id précis
subgraphs	défaut (False)	True : inclut l'état des sous-graphes en cours
↩ .next	(), ('valider',)	nœuds à exécuter ensuite ; () = rien à faire
↩ .metadata	["step"]	step, source ("input", "loop", "update"…)
↩ .interrupts	HITL	pauses en cours : Interrupt(value, id)

⚠️ Pièges

- Graphe compilé sans checkpointer : ValueError « No checkpointer set »
- Thread inconnu : un état vide, pas d'erreur : vérifier le thread_id
- Modifier .values ne change pas le checkpoint : passer par update_state

🔎 get_state_history : tous les checkpoints d'un thread

La liste des sauvegardes, du plus récent au plus ancien : on y choisit d'où repartir

📦 langgraph 1.2

```python
historique = list(graphe.get_state_history(config_A))
for snap in historique:                  # le plus récent d'abord
    print(snap.metadata["step"], snap.next)
avant = [h for h in historique if h.next == ("garde_fou",)][-1]
```

À quoi ça sert

Parcourt tous les checkpoints d'un thread, du plus récent au plus ancien, chacun sous forme de StateSnapshot. On y trouve le point exact d'où rejouer ou bifurquer : son .config contient le checkpoint_id. Un tour de conversation produit plusieurs checkpoints : un par super-step, plus l'entrée brute (step -1) et le départ (__start__).

↩️ Renvoie

Un itérateur (générateur) de StateSnapshot : list(...) pour l'indexer, le parcourir deux fois ou le filtrer.

Paramètre	Dans le cours	Pourquoi · autres choix
`config`	`config_A`	le thread à parcourir
`filter`	non utilisé	filtre sur metadata, ex. {"source": "update"} ou {"step": 3}
`before`	non utilisé	config d'un checkpoint : seulement les plus anciens (pagination)
`limit`	non utilisé	nombre maximal de snapshots, pour les longues conversations
↩ snap.config	`avant.config`	à passer à invoke(None, ...) ou update_state(...)

⚠️ Pièges

- Ordre inverse : [0] est le plus récent, [-1] le plus ancien
- Un itérateur se consomme une fois : le garder dans une liste
- Plusieurs snapshots ont le même next (un par tour) : [-1] = le premier tour

🔎 SqliteSaver : des checkpoints dans un fichier

Même contrat que InMemorySaver ; seule la durée de vie change

📦 langgraph-checkpoint-sqlite 3.1

```
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver

connexion = sqlite3.connect(chemin, check_same_thread=False)
saver = SqliteSaver(connexion)
```

À quoi ça sert

Un checkpointer qui écrit dans un fichier SQLite : la conversation survit au redémarrage du kernel ou du service. Les tables checkpoints et writes sont créées au premier usage. On le passe à compile(checkpointer=…) comme InMemorySaver. Pour un graphe async (jalon J2, J2-05), la variante AsyncSqliteSaver prend une connexion aiosqlite.

↩️ Renvoie

Un SqliteSaver (BaseCheckpointSaver) pour compile(checkpointer=…). Il offre aussi list et delete_thread (slide « purge »).

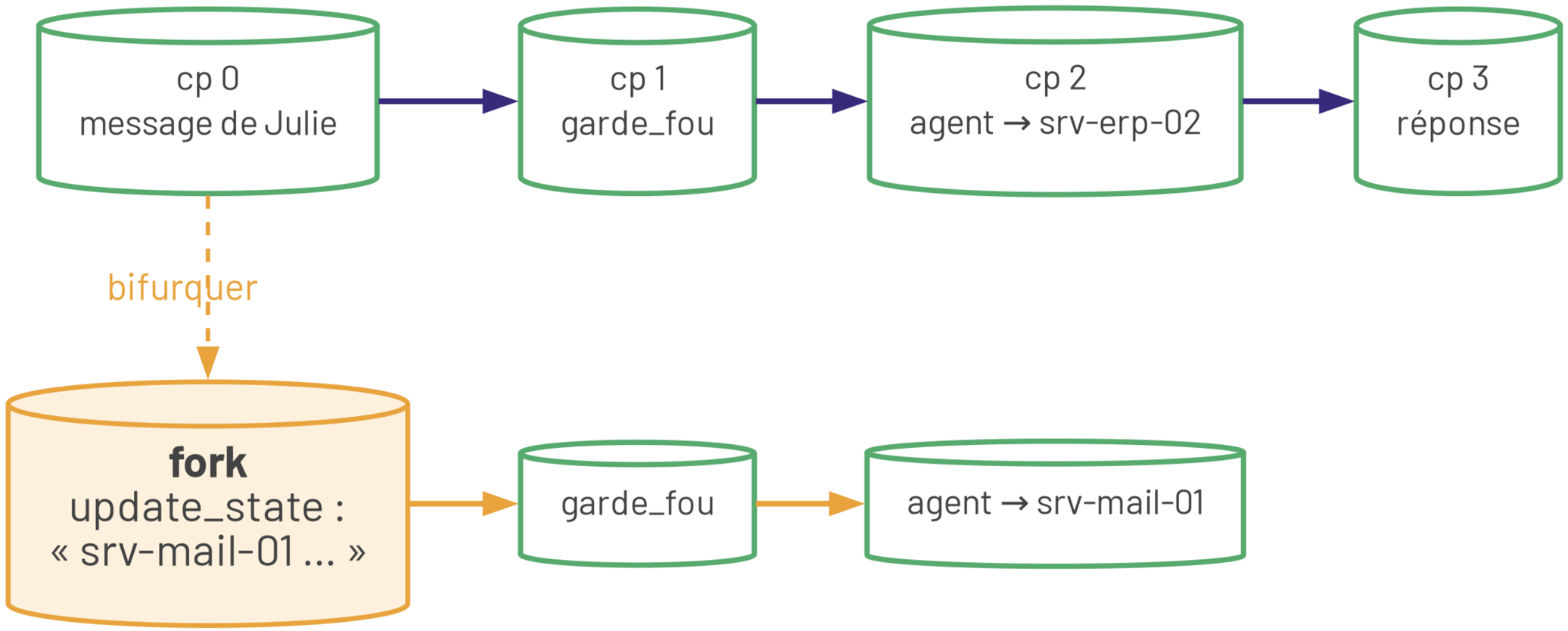
Paramètre	Dans le cours	Pourquoi · autres choix
`conn`	`sqlite3.connect(...)`	une connexion ouverte par vous : vous gérez sa durée de vie
`check_same_thread`	`False`	option de sqlite3 (défaut True) : LangGraph écrit depuis d'autres threads
`serde`	défaut	sérialiseur ; EncryptedSerializer existe pour chiffrer les états
`from_conn_string`	non utilisé	with SqliteSaver.from_conn_string("f.sqlite") as saver : ouvre ET ferme
`AsyncSqliteSaver`	jalon J2, J2-05	langgraph.checkpoint.sqlite.aio ; conn = await aiosqlite.connect(…)
`chemin_checkpoints(nom)`	`"j2.sqlite"`	helper du cours : data/checkpoints/, créé au besoin, ignoré par Git

⚠️ Pièges

- SqliteSaver + ainvoke : NotImplementedError, prendre AsyncSqliteSaver
- Le fichier contient les conversations en clair : hors Git, droits restreints
- Un fichier = un poste : plusieurs instances ⟶ PostgresSaver

Voyage dans le temps : rejouer et bifurquer

Rejouer : invoke(None, cp) · Bifurquer : update_state(cp, valeurs)



Bifurquer : corriger un message et relancer

```
historique = list(support.get_state_history(config))
avant = [h for h in historique if h.next == ("garde_fou",)][-1]
origine = avant.values["messages"][-1]

# même id → add_messages REMPLACE le message (vu en J1-03)
corrige = HumanMessage("srv-mail-01 ne répond plus", id=origine.id)
fork = support.update_state(avant.config, {"messages": [corrige]})
support.invoke(None, fork)          # relance depuis la branche
```

⚠️ Rejouer n'est pas gratuit

Les nœuds relancés rappellent le LLM… et les outils.

Un outil qui écrit, rejoué deux fois = un doublon.

→ **idempotence**

🔎 update_state : écrire une branche

Un nouveau checkpoint, comme si un nœud avait renvoyé ces valeurs

```
📦 langgraph 1.2

fork = graphe.update_state(avant.config,
                           {"messages": [corrige]})
graphe.invoke(None, fork)        # exécute la branche
```

À quoi ça sert

Écrit un nouveau checkpoint à partir de celui désigné par le config, comme si un nœud avait renvoyé ces valeurs : elles passent donc par les reducers (add_messages remplace un message de même id). L'historique d'origine reste intact : on crée une branche. Sert à corriger une entrée, débloquer un graphe ou rejouer un scénario.

↩️ Renvoie

Le RunnableConfig du nouveau checkpoint (thread_id, checkpoint_ns, checkpoint_id) : à passer à invoke(None, …) pour exécuter la branche.

Paramètre	Dans le cours	Pourquoi · autres choix
`config`	`avant.config`	le checkpoint d'où bifurquer ; le config du thread seul = le dernier
`values`	`{"messages": [corrige]}`	mise à jour partielle, comme le retour d'un nœud
`as_node`	défaut	nœud « auteur » de l'écriture, il décide de la suite ; défaut : le dernier nœud qui a écrit, s'il n'est pas ambigu
`task_id`	non utilisé	cas avancé : rattacher l'écriture à une tâche précise
`invoke(None, fork)`	juste après	None = pas de nouvelle entrée : on reprend depuis la branche

⚠️ Pièges

- Un id de message différent : add_messages AJOUTE au lieu de remplacer
- invoke(None, avant.config) au lieu de fork : on rejoue l'original
- Relancer rappelle le LLM et les outils : idempotence

Une purge qui s'applique vraiment

```python
from datetime import timedelta
from helpdesk_ops.persistance import purger_threads

# 1. simulation : à journaliser avant la première vraie purge
purger_threads(saver, timedelta(days=30), simulation=True)

# 2. en tâche planifiée (cron), durée écrite au registre des traitements
purger_threads(saver, timedelta(days=30))

# 3. demande d'effacement d'une personne
saver.delete_thread("j2-01-julie")
```

Effacée partout ?

- checkpoints : ✅
- traces Langfuse : masquées, mais le nom passait (J1)
- tickets : le résumé
- sauvegardes, journaux applicatifs

🔎 delete_thread et list : effacer, inventorier

Les méthodes du checkpointer lui-même, sans passer par le graphe

📦 langgraph-checkpoint 4.2 · sqlite 3.1

```python
saver.delete_thread("j2-01-julie")        # droit à l'effacement
for cp in saver.list(None):               # tous les threads
    cp.config["configurable"]["thread_id"], cp.checkpoint["ts"]
```

À quoi ça sert

delete_thread efface tout ce qui appartient à un thread : checkpoints et écritures intermédiaires. list énumère les checkpoints d'un thread, ou de tous avec None. purger_threads du cours combine les deux : list pour trouver la dernière activité de chaque conversation (checkpoint["ts"]), puis delete_thread pour celles qui dépassent la durée de conservation.

↩️ Renvoie

delete_thread : None. list : un itérateur de CheckpointTuple (config, checkpoint, metadata, parent_config, pending_writes).

Paramètre	Dans le cours	Pourquoi · autres choix
`delete_thread(id)`	`"j2-01-julie"`	checkpoints ET writes ; aucune erreur si le thread n'existe pas
`list(config)`	`None`	None : tous les threads ; un config : un seul thread
`filter, before, limit`	non utilisé	filtre sur metadata, pagination, nombre maximal
`prune(ids, strategy=)`	non utilisé	défaut "keep_latest" ; SqliteSaver 3.1 : NotImplementedError
`purger_threads(…)`	`timedelta(days=30)`	helper du cours ; simulation=True liste sans supprimer

⚠️ Pièges

- L'effacement ne touche ni Langfuse, ni le ticketing, ni les sauvegardes
- prune figure dans l'API mais pas dans SqliteSaver
- Une purge sans simulation préalable : rien à journaliser, rien à vérifier

Lire les messages d'une conversation

L'état d'un agent est une liste de messages typés : on la filtre comme une liste Python

```python
from langchain_core.messages import AIMessage, HumanMessage, ToolMessage

messages = avec_memoire.get_state(config_A).values["messages"]
print([type(m).__name__ for m in messages])

# exemple : les questions posées par l'utilisateur
questions = [m.content for m in messages if isinstance(m, HumanMessage)]
print("questions :", questions)
```

▶ **Sortie exacte (modèle factice, aucun outil appelé)**

```
['HumanMessage', 'AIMessage', 'HumanMessage', 'AIMessage']
questions : ['Le serveur srv-erp-02 est-il en ligne ?', 'Et ce serveur, son nom exact ?']
```

Message	Repérer
HumanMessage	l'utilisateur
AIMessage	la réponse… ou une demande d'outil (m.tool_calls non vide)
ToolMessage	le résultat d'un outil

🧰 Boîte à outils du mini-défi J2-01

Droit d'accès (RGPD art. 15) : rendre à Julie une copie lisible de sa conversation

Besoin	Syntaxe	Revoir
Une conversation de 2 tours	demander(support, question, thread_id="j2-01-defi-mp")(×2)	Trois checkpointers
Les messages du thread	`graphe.get_state({"configurable": {"thread_id": t}}).values["messages"]`	Lire les messages
Garder l'utilisateur	`isinstance(m, HumanMessage)`	Lire les messages
Garder les réponses, pas les appels	`isinstance(m, AIMessage) and not m.tool_calls`	Lire les messages
Construire le résultat	`[{"role": "utilisateur", "contenu": m.content}, …]`	Python (liste de dicts)

Discussion : faut-il aussi rendre les résultats d'outils ? les décisions du garde-fou ? Ce sont des données qui concernent Julie.

🧪 À vous : J2-01

08h30–09h45 · notebook J2_01_persistance

- **§2** l'agent amnésique : deux questions sans checkpointer
- **§3** InMemorySaver : deux conversations A et B, isolées par thread_id
- **§4** get_state et get_state_history : compter les checkpoints
- **§5** SqliteSaver : la conversation survit au « redémarrage »
- **§6 – 7** bifurquer depuis un checkpoint, puis chercher l'e-mail en clair et effacer
- **Mini-défi :** exporter_conversation(graphe, thread_id), le droit d'accès

Slides à revoir

- trois checkpointers
- lire l'état · bifurquer
- RGPD · purge

Le fil rouge

construire_support(llm, checkpointer=…) : l'agent du jalon J1, rendu stateful.

Étape 4 : le code ajouté (1/3)

le code ajouté de etape_03_stategraph.py à etape_04_memoire.py

```
# ▼ ligne 19 · remplace : import sys
import argparse

# ▼ ligne 27
from langgraph.checkpoint.sqlite import SqliteSaver                    # ═══ NOUVEAU (étape 4)

# ▼ ligne 36 · remplace : TICKET = sys.argv[1] if len(sys.argv) > 1 else "srv-erp
# ═══ NOUVEAU (étape 4) : la ligne de commande désigne la conversation ═══
parser = argparse.ArgumentParser(description="HelpDesk-Ops, étape 4 : la mémoire")
parser.add_argument("message", nargs="?", default="Mon VPN ne marche plus depuis ce matin")
parser.add_argument("--thread", default="conv-demo", help="identifiant de la conversation")
parser.add_argument("--oublier", action="store_true", help="effacer la conversation (RGPD)")
args = parser.parse_args()


# ▼ ligne 45
BASE = RACINE / "data" / "checkpoints" / "construction.sqlite"     # ═══ NOUVEAU (étape 4) : ignoré par Git

# ▼ ligne 151 · remplace : dessiner(graphe, "etape_03_graphe")
# ═══ NOUVEAU (étape 4) : compiler AVEC le checkpointer ; config["configurable"]["thread_id"] = la conversation ═══
BASE.parent.mkdir(parents=True, exist_ok=True)
with SqliteSaver.from_conn_string(str(BASE)) as memoire:
    graphe = builder.compile(checkpointer=memoire)
    config = {"configurable": {"thread_id": args.thread}, "callbacks": [handler],
              "run_name": "helpdesk", "recursion_limit": 10}
```

Étape 4 : le code ajouté (2/3)

le code ajouté de etape_03_stategraph.py à etape_04_memoire.py

```
# ▼ (suite)
    if args.oublier:                                          # ═══ NOUVEAU (étape 4) : droit à l'effacement
        memoire.delete_thread(args.thread)
        print(f"🧹 Conversation {args.thread} effacée du fichier SQLite.")
        raise SystemExit(0)

    avant = len(graphe.get_state(config).values.get("messages", []))
    if avant == 0:
        dessiner(graphe, "etape_04_graphe")
    print(f"💾 Thread {args.thread} : {avant} message(s) déjà en mémoire")
    print(f"🎫 {args.message}")

    with propagate_attributes(session_id=args.thread, user_id="agent-n1-042",
                              tags=["construction", "etape-4"]):
        etat = graphe.invoke({"ticket": args.message}, config=config)

    for appel in (a for m in etat["messages"][avant:] for a in getattr(m, "tool_calls", [])):
        print(f"   🧰 outil appelé : {appel['name']}({appel['args']})")
    print(f"   🤖 {etat['messages'][-1].content}")

# ▼ ligne 178 · remplace : with propagate_attributes(session_id="construction-etap (+15 ligne(s))
    historique = list(graphe.get_state_history(config))
    print(f"💾 Thread {args.thread} : {len(etat['messages'])} messages, {len(historique)} checkpoints "
          f"dans {BASE.relative_to(RACINE)}")
```

🖇️ Étape 4 : le code ajouté (3/3)

le code ajouté de etape_03_stategraph.py à etape_04_memoire.py

```
# ▼ ligne 183 · remplace : print("🔭 Langfuse › Tracing › helpdesk (enfants : trier
print(f"🔭 Langfuse › Sessions › {args.thread}")
```

🖇️ Human-in-the-loop

Fiches de référence, dans l'ordre des slides présentées

p. 23 🔎 interrupt : suspendre pour une décision humaine
p. 24 🔎 Command : reprendre, mettre à jour, router
p. 25 🔎 RunnableConfig : le config de l'appel, dans le nœud
p. 26 Des décisions riches, et validées
p. 27 HITL dans create_agent : le HumanInTheLoopMiddleware
p. 28 🔎 HumanInTheLoopMiddleware : valider les appels d'outils
p. 29 🧪 À vous : J2-02
p. 30 🧰 Boîte à outils de l'exercice 2.1
p. 31 🧪 Exercice 2.1 : un ticket P1 exige une approbation
p. 32 🖇️ Étape 5 : le code ajouté (1/5)
p. 33 🖇️ Étape 5 : le code ajouté (2/5)
p. 34 🖇️ Étape 5 : le code ajouté (3/5)
p. 35 🖇️ Étape 5 : le code ajouté (4/5)
p. 36 🖇️ Étape 5 : le code ajouté (5/5)

🔎 interrupt : suspendre pour une décision humaine

Une fonction appelée DANS un nœud : arrêt, sauvegarde, puis reprise avec la réponse

📦 langgraph 1.2

```python
from langgraph.types import interrupt

def valider(state):
    reponse = interrupt({"question": "Créer le ticket P1 ?"})
    return {"decision": reponse}
```

À quoi ça sert

Au premier passage, interrupt() arrête le graphe, enregistre un checkpoint et rend la valeur à l'appelant de invoke. À la reprise (Command(resume=…), même thread), le nœud repart DU DÉBUT ; cette fois interrupt() ne s'arrête plus et renvoie la valeur de la reprise. C'est la brique de toute validation humaine du cours : J2-02, Ex2.1, jalon J2.

↩️ Renvoie

1er passage : ne revient pas (le nœud s'arrête) ; l'appelant reçoit Interrupt(value, id) dans .interrupts (v2). À la reprise : la valeur passée à Command(resume=…).

Paramètre	Dans le cours	Pourquoi · autres choix
`value`	`{"question": …}`	ce que verra l'humain : un dict sérialisable (question, ticket proposé, actions permises)
`response_schema`	non utilisé	modèle Pydantic de la réponse attendue : reprise invalide ⟶ ValidationError
checkpointer	`InMemorySaver()`	sans lui, l'arrêt a lieu, mais la reprise lève RuntimeError
plusieurs interrupt()	boucle des décisions riches	associés aux reprises dans l'ordre d'appel, au sein du nœud

⚠️ Pièges

- Tout code AVANT interrupt() s'exécute deux fois
- try/except Exception autour d'interrupt() avale l'arrêt : GraphInterrupt hérite d'Exception
- Un autre thread_id à la reprise : la pause n'est pas retrouvée

🔎 Command : reprendre, mettre à jour, router

Un ordre adressé au graphe : passé à invoke, ou renvoyé par un nœud

📦 langgraph 1.2

```python
from langgraph.types import Command

graphe.invoke(Command(resume="approuvé"), config)        # reprise
return Command(update={"validation": r}, goto="creer")  # nœud
def noeud(state) -> Command[Literal["creer", "refuser"]]:
```

À quoi ça sert

Passé à invoke, Command(resume=…) répond à l'interruption en attente du thread. Renvoyé par un nœud, Command(update=…, goto=…) met à jour l'état ET choisit le nœud suivant, sans arête conditionnelle : la décision et le routage vivent dans le même code (décisions riches, qualifier, error_handler). L'annotation Command[Literal[…]] déclare les destinations pour le dessin du graphe.

↩️ Renvoie

Un objet Command : il ne fait rien seul. Passé à invoke (resume), ou renvoyé par un nœud (update, goto).

Paramètre	Dans le cours	Pourquoi · autres choix
`resume`	`"approuvé…", {"action": …}`	pour invoke : devient la valeur renvoyée par interrupt() ; n'importe quelle valeur sérialisable
`update`	`{"validation": reponse}`	comme le dict renvoyé par un nœud : passe par les reducers
`goto`	`"creer", END`	nœud suivant ; aussi une liste de nœuds, ou des Send (J3)
`graph`	défaut (None)	None : graphe courant ; Command.PARENT : depuis un sous-graphe, router dans le parent
`Command[Literal]`	annotation de retour	destinations du dessin ; add_node(destinations=…) fait de même

⚠️ Pièges

- goto vers un nom inconnu : simple avertissement, et le graphe s'arrête
- Command(resume=…) sur un thread terminé : rien n'est relancé

🔎 RunnableConfig : le config de l'appel, dans le nœud

Le même dict que le 2e argument d'invoke, injecté par LangGraph

📦 langchain-core 1.6

```python
from langchain_core.runnables import RunnableConfig

def creer_ticket(state, config: RunnableConfig) -> dict:
    thread_id = config["configurable"]["thread_id"]
```

À quoi ça sert

RunnableConfig est le type (un TypedDict) du config passé à invoke. Quand un nœud déclare un paramètre nommé config, LangGraph lui injecte le config de l'appel en cours. Le nœud connaît alors sa conversation : le thread_id sert de clé d'idempotence (Ex2.1, jalon J2), les callbacks et métadonnées gardent le traçage groupé.

↩️ Renvoie

Rien à appeler : c'est un dict. config["configurable"]["thread_id"] lève KeyError si l'appel n'en a pas fourni.

Paramètre	Dans le cours	Pourquoi · autres choix
nom du paramètre	`config`	c'est le NOM qui déclenche l'injection ; l'annotation RunnableConfig est conseillée (autre type : avertissement)
`configurable`	`["thread_id"]`	valeurs d'exécution : thread_id, checkpoint_id… et vos propres clés
`callbacks`	[CallbackHandler()] (J1)	transmis aux appels LLM du nœud si on lui passe config
`metadata / tags`	non utilisé ici	étiquettes reprises dans les traces
`run_name`	non utilisé	nom de l'exécution dans les traces
`recursion_limit`	défaut	limite de super-steps du graphe

⚠️ Pièges

- def noeud(state, cfg: RunnableConfig) : rien n'est injecté, TypeError
- Clé d'idempotence en uuid4 : chaque rejeu en crée une nouvelle, donc un doublon
- Graphe invoqué sans thread_id : KeyError dans le nœud

Des décisions riches, et validées

```python
def validation_humaine(state) -> Command[Literal["creer", "refuser"]]:
    question = {"ticket_propose": {...}, "actions": ACTIONS}
    while True:          # chaque interrupt() attend sa propre reprise
        reponse = interrupt(question)
        if reponse.get("action") in ACTIONS and reponse.get("par"):
            break
        question = {**question, "erreur": f"réponse invalide : {reponse!r}"}
    if reponse["action"] == "rejeter":
        return Command(update={"validation": reponse}, goto="refuser")
    maj = {"validation": reponse}
    if reponse["action"] == "modifier":
        maj["priorite"] = reponse["priorite"]
    return Command(update=maj, goto="creer")
```

💡 Le « Tips » du Formateur

La réponse humaine vient d'une interface, d'un mail, d'un chat : c'est une ENTRÉE. On la valide comme les autres.

HITL dans create_agent : le HumanInTheLoopMiddleware

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    model=llm, tools=[statut_serveur, creer_ticket],
    checkpointer=InMemorySaver(),
    middleware=[HumanInTheLoopMiddleware(
        interrupt_on={"creer_ticket": True, "statut_serveur": False})],
)
s = agent.invoke(entree, config, version="v2")
s.interrupts[0].value["action_requests"]   # nom + arguments exacts

agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

Décision
approve
edit
reject
respond

🔎 HumanInTheLoopMiddleware : valider les appels d'outils

Le HITL de create_agent, sans écrire de nœud : interrupt() avant les outils choisis

📦 langchain 1.4.2

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

HumanInTheLoopMiddleware(interrupt_on={"creer_ticket": True})
Command(resume={"decisions": [{"type": "approve"}]})
```

À quoi ça sert

Un middleware de create_agent qui s'intercale juste avant l'exécution des outils. Si le modèle demande un outil listé, l'agent appelle interrupt() avec le nom et les arguments exacts de chaque appel ; l'humain répond par une décision par appel. Même mécanique qu'au §3, sans écrire le nœud : il faut donc un checkpointer (create_agent(checkpointer=...)) et un thread_id.

↩️ Renvoie

L'invoke s'arrête : .interrupts[0].value = {"action_requests": [{name, args, description}], "review_configs": [...]}. La reprise attend {"decisions": [...]}, une par appel, dans l'ordre.

Paramètre	Dans le cours	Pourquoi · autres choix
`interrupt_on`	`{"creer_ticket" : True, …}`	True : 4 décisions ; False ou absent : sans validation
InterruptOnConfig	non utilisé	au lieu de True : allowed_decisions, description, when
`description_prefix`	défaut	texte de la demande
`edit_notice`	défaut	note ajoutée au résultat d'un appel modifié
décision edit	non utilisé	{"type": "edit", "edited_action": {"name": ..., "args": {...}}}
`reject / respond`	non utilisé	reject : outil non exécuté ; respond : l'humain répond à sa place

⚠️ Pièges

- Pas de checkpointer dans create_agent : pas de reprise
- Outil non appelé par le modèle : aucune pause
- Décisions ≠ appels en attente : ValueError

🧪 À vous : J2-02

10h00–12h00 · notebook J2_02_human_in_the_loop, puis l'exercice 2.1

- **§3** suspendre et reprendre le mini-graphe (sans LLM)
- **§4** provoquer le piège du rejeu, puis l'idempotence du ticketing
- **§5** approuver, modifier, rejeter ; une réponse invalide repose la question
- **§6** HumanInTheLoopMiddleware sur un create_agent
- **Exercice 2.1 :** un ticket P1 exige une approbation (45 min)

Slides à revoir

- interrupt() 1/2 et 2/2
- piège du rejeu · ticketing
- décisions riches

👀 Dans Langfuse

Une validation = deux traces dans la même session : l'arrêt, puis la reprise.

🧰 Boîte à outils de l'exercice 2.1

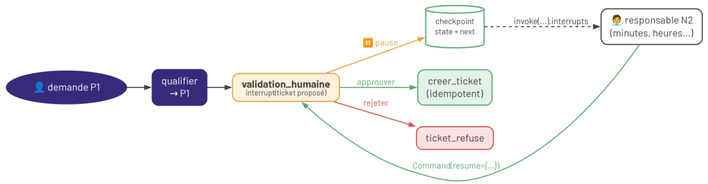
Chaque étape de l'énoncé, sa syntaxe, et la slide où elle a été vue

Étape	Syntaxe	Revoir
Fourni	qualifier : classifieur LLM + estimer_priorite (règle du défi J1)	énoncé, cellule 3
Router P1 / autres	`def router_priorite(state) -> Literal["validation_humaine", "creer_ticket"]:`	J1 · premier graphe
Suspendre	reponse = interrupt({"ticket_propose": {…}}) en 1re ligne utile du nœud	interrupt() 1/2
Décider la suite	`return Command(update={"validation": reponse}, goto="ticket_refuse")`	Décisions riches
Lire le thread_id	def creer_ticket(state, config: RunnableConfig): puis config["configurable"]["thread_id"]	config: RunnableConfig
Créer sans doublon	`ticketing.creer(cle_idempotence=thread_id, …, valide_par=…)`	Ticketing simulé
Compiler	`b.compile(checkpointer=InMemorySaver())`	Trois checkpointers
Lancer, voir la pause	`s = circuit.invoke({"ticket": …}, config_de("a"), version="v2"); s.interrupts`	interrupt() 1/2
Approuver / rejeter	`circuit.invoke(Command(resume={"action": "approuver", "par": "resp-n2"}), config_de("a"))`	interrupt() 2/2
Rejouer la création	`avant = [h for h in circuit.get_state_history(cfg) if h.next == ("creer_ticket",)][-1] ; circuit.invoke(None, avant.config)`	Bifurquer

🧪 Exercice 2.1 : un ticket P1 exige une approbation

exercices/enonces/Ex2_1_validation_p1.ipynb · 45 min

- **Fourni :** qualifier (catégorie par LLM, priorité par la règle du défi J1)
- **Routage :** P1 ⟶ validation_humaine, sinon ⟶ creer_ticket
- **Validation :** interrupt(), réponse {action, par, motif}
- **Création :** nœud séparé, clé d'idempotence = thread_id
- **Tests :** 3 demandes, une P1 approuvée, une P1 rejetée, une P3
- **Rejeu :** rejouer creer_ticket depuis son checkpoint : un seul ticket



Étape 5 : le code ajouté (1/5)

le code ajouté de etape_04_memoire.py à etape_05_validation_ticket.py

```
# ▼ ligne 24
import json

# ▼ ligne 26
from typing import Literal

# ▼ ligne 28 · remplace : from langchain_core.messages import HumanMessage, Syste
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage

# ▼ ligne 31
from langchain_core.runnables import RunnableConfig                    # ═══ NOUVEAU (étape 5)

# ▼ ligne 37
from langgraph.types import Command, interrupt                         # ═══ NOUVEAU (étape 5)

# ▼ ligne 44 · remplace : parser = argparse.ArgumentParser(description="HelpDesk- (+1 ligne(s))
parser = argparse.ArgumentParser(description="HelpDesk-Ops, étape 5 : validation humaine et ticket")
parser.add_argument("message", nargs="?", default="Le VPN est bloqué pour tout le service, urgent")

# ▼ ligne 48
parser.add_argument("--approuver", metavar="NOM", help="valider le ticket P1 en attente")      # ═══ NOUVEAU (étape 5)
parser.add_argument("--refuser", metavar="NOM", help="refuser le ticket P1 en attente")       # ═══ NOUVEAU (étape 5)
parser.add_argument("--motif", default="", help="motif du refus")                             # ═══ NOUVEAU (étape 5)
```

Étape 5 : le code ajouté (2/5)

le code ajouté de etape_04_memoire.py à etape_05_validation_ticket.py

```
# ▼ ligne 56
TICKETS = RACINE / "data" / "checkpoints" / "construction_tickets.json"   # ═══ NOUVEAU (étape 5) : ignoré par Git

# ▼ ligne 123
def enregistrer_ticket(cle: str, **champs) -> tuple[dict, bool]:
    """Renvoie (ticket, créé). La même clé renvoie le ticket existant : pas de doublon au rejeu."""
    tickets = json.loads(TICKETS.read_text(encoding="utf-8")) if TICKETS.exists() else {}
    if cle in tickets:
        return tickets[cle], False
    ticket = {"id": f"NT-{len(tickets) + 1:04d}", **champs}
    tickets[cle] = ticket
    TICKETS.write_text(json.dumps(tickets, ensure_ascii=False, indent=2), encoding="utf-8")
    return ticket, True


# ---------------------------------------------------------------------- l'état et les nœuds

# ▼ ligne 140
    valide_par: str          # ═══ NOUVEAU (étape 5)

# ▼ ligne 147 · remplace : return {"categorie": categorie, "priorite": priorite, "
    return {"categorie": categorie, "priorite": priorite, "file": file_attente(categorie), "valide_par": "",
```

📎 Étape 5 : le code ajouté (3/5)

le code ajouté de etape_04_memoire.py à etape_05_validation_ticket.py

```python
# ▼ ligne 155
# ═══ NOUVEAU (étape 5) : après l'agent, trois chemins au lieu de deux ═══
def apres_agent(state: EtatTicket) -> Literal["tools", "valider", "creer_ticket"]:
    if tools_condition(state) == "tools":
        return "tools"
    return "valider" if state["priorite"] == "P1" else "creer_ticket"


# ═══ NOUVEAU (étape 5) : la pause humaine. À la reprise, le nœud repart DU DÉBUT : rien avant interrupt() ═══
def valider(state: EtatTicket) -> Command[Literal["creer_ticket", "__end__"]]:
    decision = interrupt({"question": "Créer ce ticket P1 ?", "ticket": state["ticket"],
                          "categorie": state["categorie"], "file": state["file"]})
    if decision["action"] == "approuver":
        return Command(goto="creer_ticket", update={"valide_par": decision["par"]})
    refus = AIMessage(f"Ticket P1 refusé par {decision['par']}" + (f" : {decision['motif']}" if decision["motif"] else "."))
    return Command(goto=END, update={"messages": [refus]})


# ═══ NOUVEAU (étape 5) : l'action à impact, dans son propre nœud ═══
def creer_ticket(state: EtatTicket, config: RunnableConfig) -> dict:
    cle = f"{config['configurable']['thread_id']}:{normaliser(state['ticket'])}"
    ticket, cree = enregistrer_ticket(cle, categorie=state["categorie"], priorite=state["priorite"],
                                      file=state["file"], resume=state["ticket"][:120],
                                      valide_par=state["valide_par"] or None)
    texte = f"Ticket {ticket['id']} {'créé' if cree else 'déjà existant'} dans la file {ticket['file']}"
    return {"messages": [AIMessage(texte + (f", validé par {ticket['valide_par']}." if ticket["valide_par"] else "."))]}
```

📎 Étape 5 : le code ajouté (4/5)

le code ajouté de etape_04_memoire.py à etape_05_validation_ticket.py

```
# ▼ ligne 186
builder.add_node("valider", valider)                    # ═══ NOUVEAU (étape 5)
builder.add_node("creer_ticket", creer_ticket)          # ═══ NOUVEAU (étape 5)

# ▼ ligne 190 · remplace : builder.add_conditional_edges("agent", tools_condition)
builder.add_conditional_edges("agent", apres_agent)     # ═══ NOUVEAU (étape 5) : remplace tools_condition seul

# ▼ ligne 192
builder.add_edge("creer_ticket", END)                   # ═══ NOUVEAU (étape 5)

# ▼ ligne 205
# ═══ NOUVEAU (étape 5) : la question posée à l'humain, lue dans le checkpoint ═══
def en_attente(graphe, config) -> list[dict]:
    return [i.value for t in graphe.get_state(config).tasks for i in t.interrupts]



# ▼ ligne 218 · remplace : print(f"🧹 Conversation {args.thread} effacée du fichier
        print(f"🧹 Conversation {args.thread} effacée du fichier SQLite (les tickets créés restent : c'est l'ITSM).")

# ▼ ligne 223 · remplace : with propagate_attributes(session_id=args.thread, user_ (+2 ligne(s))
    if args.approuver or args.refuser:
        if not en_attente(graphe, config):
            raise SystemExit(f"Rien à valider sur le thread {args.thread}.")
        entree = Command(resume={"action": "approuver" if args.approuver else "refuser",
                                 "par": args.approuver or args.refuser, "motif": args.motif})
```

Étape 5 : le code ajouté (5/5)

le code ajouté de etape_04_memoire.py à etape_05_validation_ticket.py

```
# ▼ (suite)
        print(f"✍️  Décision de {args.approuver or args.refuser} : {'approuvé' if args.approuver else 'refusé'}")
    else:
        if en_attente(graphe, config):
            raise SystemExit(f"Le thread {args.thread} attend une validation : --approuver NOM ou --refuser NOM.")
        if avant == 0:
            dessiner(graphe, "etape_05_graphe")
        entree = {"ticket": args.message}
        print(f"🎫 {args.message}")

    with propagate_attributes(session_id=args.thread, user_id="agent-n1-042", tags=["construction", "etape-5"]):
        etat = graphe.invoke(entree, config=config)

# ▼ ligne 242 · remplace : print(f"   🤖 {etat['messages'][-1].content}") (+3 ligne(s))
    attente = en_attente(graphe, config)
    if attente:
        q = attente[0]
        print(f"   🏷️  {etat['categorie']} · {etat['priorite']} · file {etat['file']}")
        print(f"   🤖 {etat['messages'][-1].content}")
        print(f"⏸  {q['question']} Le graphe est arrêté, son état est dans le checkpoint. Pour reprendre :")
        print(f"   ... etape_05_validation_ticket.py --thread {args.thread} --approuver <nom>   (ou --refuser <nom> --motif ...)")
    else:
        print(f"   🎫 {etat['messages'][-1].content}")
        print(f"📒 Tickets : {TICKETS.relative_to(RACINE)}")
```

📎 Sortie structurée

Fiches de référence, dans l'ordre des slides présentées

p. 38 🔎 BaseModel, Field, Literal : écrire le contrat
p. 39 Mesures de préparation
p. 40 🔎 with_structured_output : un LLM qui remplit le contrat
p. 41 La réponse structurée d'un agent : response_format
p. 42 🔎 ToolStrategy : la réponse structurée d'un agent
p. 43 Router sur les champs : Command(goto=…)
p. 44 💉 À vous : J2-03
p. 45 📎 Étape 6a : le code ajouté (1/3)
p. 46 📎 Étape 6a : le code ajouté (2/3)
p. 47 📎 Étape 6a : le code ajouté (3/3)

🔎 BaseModel, Field, Literal : écrire le contrat

Une classe Pydantic = un schéma JSON qui contraint le LLM + un validateur pour la réponse

📦 pydantic 2.13

```python
from typing import Literal
from pydantic import BaseModel, Field, ValidationError

class TicketDecision(BaseModel):
    priorite: Literal["P1", "P2", "P3", "P4"] = Field(description="…")
```

À quoi ça sert

Une classe BaseModel décrit un contrat : des champs typés, validés à la création de l'objet. with_structured_output et create_agent la convertissent en schéma JSON (model_json_schema). En json_schema, ce schéma contraint la génération : les Literal sont respectés, mais les descriptions ne sont pas lues par le modèle (les consignes vont dans le prompt) ; en function_calling, il devient un outil et ses descriptions sont lues. La réponse du LLM est ensuite revalidée contre ce même modèle.

↩️ Renvoie

Une instance validée (decision.priorite). Sinon ValidationError, dont .errors() liste loc, msg, type, input pour chaque champ refusé.

Paramètre	Dans le cours	Pourquoi · autres choix
`Literal[...]`	catégories, priorités	devient un « enum » : seules ces valeurs passent ; str laisserait passer « urgent »
`Field(description=)`	chaque champ	documente le champ ; lue en function_calling, PAS en json_schema
`Field(default=)`	non utilisé	rend le champ facultatif ; sans défaut, il est obligatoire (required)
`ge, le, max_length`	non utilisé	bornes numériques, longueur maximale d'une chaîne
docstring	« Qualification d'une… »	devient la description du schéma (même règle)
`model_dump()`	`decision.model_dump()`	objet → dict, pour le ranger dans l'état du graphe

⚠️ Pièges

- « réseau » ≠ « reseau » : un accent suffit à être refusé
- Literal limite les valeurs, pas les intentions (ASI01)
- Un champ str libre peut recevoir des données personnelles

Mesures de préparation

granite4.1:8b · poste CPU · appels isolés · 20/10/2026

	Sans prompt système	Avec définitions des priorités
`json_schema`	9 à 20 s · format toujours valide · « souris sans fil » → reseau / P1 ❌	11 à 26 s · souris → P4 · écran qui clignote → P3 ✅
`function_calling`	15 s si l'outil est appelé · 2 cas sur 3 → None après 100 à 220 s de prose ❌	13 à 34 s · 4 cas sur 4 corrects ✅

Leçon 1

Sans définition des priorités, le modèle met P1 partout : le prompt compte plus que la méthode.

Leçon 2

Les catégories restent imparfaites (VPN → autre ou logiciel) : on mesurera en J4, sur un dataset.

🔎 with_structured_output : un LLM qui remplit le contrat

Une méthode de tout modèle de chat : renvoie un nouveau Runnable, le LLM d'origine ne change pas

📦 langchain-ollama 1.1 · langchain-openai 1.6

```
qualifieur = llm.with_structured_output(
    TicketDecision, method="json_schema", include_raw=True)
sortie = qualifieur.invoke([PROMPT_QUALIFICATION, HumanMessage(d)])
```

À quoi ça sert

Enveloppe le modèle : le schéma est transmis au serveur (génération contrainte, outil ou simple mode JSON selon method), puis la réponse est parsée et validée par Pydantic. Même API pour ChatOllama et ChatOpenAI branché sur vLLM : creer_llm() choisit selon LLM_BACKEND. Le jalon J2 l'utilise dans le nœud qualifier pour obtenir un TicketDecision au lieu de prose.

↩️ Renvoie

include_raw=False : une instance TicketDecision. include_raw=True : un dict {"raw": AIMessage, "parsed": TicketDecision ou None, "parsing_error": exception ou None}.

Paramètre	Dans le cours	Pourquoi · autres choix
schema	TicketDecision	classe Pydantic (instance en sortie) ; TypedDict ou schéma JSON (dict en sortie)
method	"json_schema"	défaut des deux paquets ; "function_calling" : None possible ; "json_mode" : sans schéma
include_raw	True	défaut False : une erreur de parsing est LEVÉE ; True : capturée dans le dict
strict	non utilisé	ChatOpenAI seulement (défaut None : non transmis) ; absent de ChatOllama
entrée de invoke	[PROMPT, HumanMessage]	comme llm.invoke : texte, liste de messages ou prompt

⚠️ Pièges

- function_calling : parsed None SANS erreur si le modèle répond en prose
- Format garanti ≠ contenu juste : le prompt fait le fond
- Mesurer en isolant les appels (file d'attente du serveur)

La réponse structurée d'un agent : response_format

```python
from langchain.agents.structured_output import ToolStrategy

agent = create_agent(
    model=llm, tools=[statut_serveur, chercher_kb],
    system_prompt=PROMPT_QUALIFICATION.content,
    response_format=ToolStrategy(TicketDecision),
)
resultat = agent.invoke({"messages": [("user", "srv-erp-02 ne répond plus")]})
resultat["structured_response"]     # TicketDecision(...)
```

ToolStrategy

- le schéma est un outil final
- l'agent utilise ses outils d'abord
- relancé si le schéma n'est pas respecté (handle_errors=True)

🔎 ToolStrategy : la réponse structurée d'un agent

create_agent(response_format=...) : l'agent agit d'abord, puis remplit le contrat

📦 langchain 1.4.2

```python
from langchain.agents.structured_output import ToolStrategy

agent = create_agent(model=llm, tools=[statut_serveur, chercher_kb],
                     response_format=ToolStrategy(TicketDecision))
agent.invoke(entree)["structured_response"]
```

À quoi ça sert

response_format donne à un agent create_agent une réponse finale structurée. ToolStrategy présente le schéma comme un outil de plus : l'agent utilise d'abord ses vrais outils, puis « appelle » le schéma pour conclure. Si les arguments ne valident pas, l'erreur lui est renvoyée et il réessaie. La réponse validée est rangée dans la clé structured_response de l'état final.

↩️ Renvoie

L'état final de l'agent : messages ET structured_response (une instance TicketDecision).

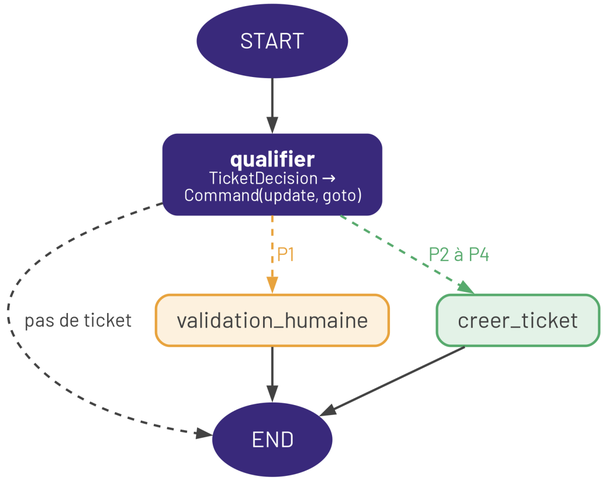
Paramètre	Dans le cours	Pourquoi · autres choix
`response_format`	`ToolStrategy(TicketDecision)`	aussi ProviderStrategy(...) ou la classe seule : stratégie choisie selon le modèle
`schema`	`TicketDecision`	Pydantic, dataclass, TypedDict ; un dict JSON n'est PAS validé
`handle_errors`	défaut (True)	renvoie l'erreur au modèle et le relance ; False : l'exception remonte ; str : message fixe
`tool_message_content`	défaut (None)	contenu du ToolMessage qui clôt l'appel du schéma
`ProviderStrategy`	non utilisé	sortie structurée native du serveur (comme json_schema), option strict

⚠️ Pièges

- Lire messages[-1].content : du texte, pas la décision
- Schéma en dict brut : handle_errors sans effet
- Sans response_format, pas de clé structured_response (KeyError)

Router sur les champs : Command(goto=...)

```python
def qualifier(state) -> Command[Literal["validation_humaine",
                                       "creer_ticket", "__end__"]]:
    decision = qualifier_prudemment(state["demande"])
    if decision is None or not decision.besoin_ticket:
        return Command(update={...}, goto=END)
    suite = ("validation_humaine" if decision.priorite == "P1"
             else "creer_ticket")
    return Command(update={"qualification": decision.model_dump()},
                   goto=suite)
```



🧪 À vous : J2-03

13h00–14h00 · notebook J2_03_sortie_structuree

- **§3** écrire TicketDecision, lire son schéma, le faire refuser « réseau » et « PO »
- **§4** qualifier trois demandes avec with_structured_output(..., include_raw=True)
- **§4** qualifier_prudemment : None quand la sortie est absente ou invalide
- **§5** create_agent(..., response_format=ToolStrategy(TicketDecision))
- **§6** router sur les champs avec Command(goto=...), sans arête conditionnelle

Slides à revoir

- le contrat · trois méthodes
- include_raw · response_format
- Command(goto)

👀 À mesurer

Les durées dépendent du matériel : notez les vôtres, sur GPU, appels isolés.

Étape 6a : le code ajouté (1/3)

le code ajouté de etape_05_validation_ticket.py à etape_06a_contrat.py

```
# ▼ ligne 38
from pydantic import BaseModel, Field                                    # ═══ NOUVEAU (étape 6a)

# ▼ ligne 41 · remplace : from helpdesk_ops.donnees import CATEGORIES, INVENTAIRE
from helpdesk_ops.donnees import INVENTAIRE, KB

# ▼ ligne 45 · remplace : parser = argparse.ArgumentParser(description="HelpDesk-
parser = argparse.ArgumentParser(description="HelpDesk-Ops, étape 6a : le contrat du tri")

# ▼ ligne 63 · remplace : PROMPT_TRI = ChatPromptTemplate.from_messages([ (+7 ligne(s))
class TicketDecision(BaseModel):
    """Qualification d'une demande de support."""
    categorie: Literal["reseau", "materiel", "logiciel", "acces", "autre"]
    priorite: Literal["P1", "P2", "P3", "P4"] = Field(
        description="P1 : production arrêtée ou nombreux utilisateurs bloqués ; P4 : simple demande")
    resume: str = Field(description="résumé du problème en une phrase, sans donnée personnelle")

# ▼ ligne 72 · remplace : def normaliser_categorie(brute: str) -> str: (+3 ligne(s))
PROMPT_TRI = ChatPromptTemplate.from_messages([
    ("system",
     "Tu es l'agent de tri N1 du help-desk de NordTech SA. Qualifie le ticket. "                    # ═══ NOUVEAU (étape 6a)
     "Priorité : P1 = service de production arrêté ou nombreux utilisateurs bloqués ; P2 = un utilisateur bloqué ; "
     "P3 = gêne sans blocage ; P4 = simple demande. Le résumé ne contient aucune donnée personnelle."),
    ("human", "Ticket : {ticket}"),
])
tri = PROMPT_TRI | llm.with_structured_output(TicketDecision, method="json_schema")   # ═══ NOUVEAU (étape 6a) : le contrat remplace le parser
```

Étape 6a : le code ajouté (2/3)

le code ajouté de etape_05_validation_ticket.py à etape_06a_contrat.py

```
# ▼ ligne 144
    resume: str              # ═══ NOUVEAU (étape 6a)

# ▼ ligne 148 · remplace : brute = tri.invoke({"ticket": state["ticket"], "categor (+2 ligne(s))
    decision = tri.invoke({"ticket": state["ticket"]})                # ═══ NOUVEAU (étape 6a) : un TicketDecision
    if decision is None:                                              # sortie inexploitable : repli prudent
        decision = TicketDecision(categorie="autre", priorite="P3", resume=state["ticket"][:120])
    # ═══ NOUVEAU (étape 6a) : la règle d'urgence est un plancher. Un format valide n'est pas une décision juste ═══
    priorite = "P1" if estimer_priorite(state["ticket"]) == "P1" else decision.priorite
    categorie = decision.categorie
    print(f"    📐 décision : {categorie} · {priorite} · « {decision.resume} »")

# ▼ ligne 156
            "resume": decision.resume,

# ▼ ligne 182 · remplace : file=state["file"], resume=state["ticket"][:120],
                                    file=state["file"], resume=state["resume"],          # ═══ NOUVEAU (étape 6a) : le résumé du contrat

# ▼ ligne 237 · remplace : dessiner(graphe, "etape_05_graphe")
            dessiner(graphe, "etape_06a_graphe")

# ▼ ligne 241 · remplace : with propagate_attributes(session_id=args.thread, user_
    with propagate_attributes(session_id=args.thread, user_id="agent-n1-042", tags=["construction", "etape-6a"]):
```

📎 Étape 6a : le code ajouté (3/3)

le code ajouté de etape_05_validation_ticket.py à etape_06a_contrat.py

```
# ▼ ligne 252 · remplace : print(f"   ... etape_05_validation_ticket.py --thread {
        print(f"   ... etape_06a_contrat.py --thread {args.thread} --approuver <nom>   (ou --refuser <nom> --motif ...)")
```

🖇️ Robustesse (démo) : pour aller plus loin

Fiches de référence, dans l'ordre des slides présentées

p. 49 RetryPolicy en code, sur une supervision simulée
p. 50 🔎 RetryPolicy : réessayer un nœud
p. 51 Que réessaie LangGraph par défaut ?
p. 52 TimeoutPolicy : uniquement sur des nœuds async
p. 53 🔎 TimeoutPolicy : borner la durée d'une tentative
p. 54 Le gestionnaire d'erreur, en code
p. 55 🔎 error_handler et NodeError : le repli
p. 56 🔎 add_node et set_node_defaults : les options de robustess
p. 57 Le piège du ToolNode qui avale les erreurs
p. 58 Le constater : handle_tool_errors=True contre le défaut
p. 59 🔎 ToolNode : le nœud qui exécute les outils
p. 60 Répondre à chaque appel d'outil en attente
p. 61 🔎 ToolMessage : la réponse à UN appel d'outil
p. 62 Sous-graphes : composer sans tout mélanger
p. 63 🔎 Un graphe compilé comme nœud : le sous-graphe
p. 64 🧪 À vous : J2-04
p. 65 🧰 Boîte à outils de l'exercice 2.2
p. 66 🧪 Exercice 2.2 : un outil réseau instable

RetryPolicy en code, sur une supervision simulée

helpdesk_ops.reseau.RESEAU : des pannes reproductibles, pour tester la robustesse

```python
from langgraph.types import RetryPolicy
from helpdesk_ops.reseau import RESEAU

async def pinger(state: Diagnostic) -> dict:          # nœud async
    return {"resultat": await RESEAU.ping(state["hote"])}

def graphe_ping(**options_noeud):                     # START -> pinger -> END
    b = StateGraph(Diagnostic)
    b.add_node("pinger", pinger, **options_noeud)      # options : retry, timeout…
    ...

RESEAU.configurer(echecs_avant_succes=2)              # 2 échecs, puis succès
retry = RetryPolicy(max_attempts=3, initial_interval=0.2,
                    backoff_factor=2.0, retry_on=ConnectionError)
print(await graphe_ping(retry_policy=retry).ainvoke({"hote": "srv-mail-01"}))
print("appels :", RESEAU.appels)
```

▶ **Sortie exacte (sans retry : ConnectionError au 1er appel)**

```
{'hote': 'srv-mail-01', 'resultat': 'srv-mail-01 : répond au ping en 3 ms', …}
appels : 3
```

RESEAU.configurer(...)

- echecs_avant_succes=n : n échecs d'abord
- taux_echec=0.5, graine=3 : pannes aléatoires reproductibles
- latence_s=3.0 : chaque appel dure 3 s
- RESEAU.appels : le compteur, remis à 0

🔎 RetryPolicy : réessayer un nœud

Une politique attachée au nœud, sans toucher à son code

📦 langgraph 1.2

```python
from langgraph.types import RetryPolicy

retry = RetryPolicy(max_attempts=3, initial_interval=0.2,
                    backoff_factor=2.0, retry_on=ConnectionError)
b.add_node("pinger", pinger, retry_policy=retry)
```

À quoi ça sert

Si le nœud lève une exception acceptée par retry_on, LangGraph attend puis relance le nœud EN ENTIER, jusqu'à max_attempts tentatives au total. Les attentes grandissent (backoff exponentiel), avec une part d'aléatoire (jitter) pour que des clients en panne ne réessaient pas tous au même instant. Après la dernière tentative : l'error_handler s'il existe, sinon l'exception remonte.

↩️ Renvoie

Un objet de configuration (NamedTuple) pour add_node(retry_policy=…) ou set_node_defaults. Une liste de politiques : la première qui correspond s'applique.

Paramètre	Dans le cours	Pourquoi · autres choix
`max_attempts`	`3`	défaut 3 : tentatives AU TOTAL, la 1re comprise
`initial_interval`	0.2 (s)	défaut 0.5 : 1re attente ; court en démo, plus long face à un vrai service
`backoff_factor`	`2.0`	défaut 2.0 : chaque attente est multipliée (0,2 → 0,4 s)
`max_interval`	défaut (128.0)	plafond d'une attente, en secondes
`jitter`	défaut (True)	ajoute de l'aléatoire aux attentes
`retry_on`	`ConnectionError`	une exception, une séquence, ou une fonction exc → bool ; défaut : default_retry_on

⚠️ Pièges

- Rejouer un nœud qui écrit : doublon sans clé d'idempotence
- Le défaut réessaie presque tout, NodeTimeoutError comprise
- Retry d'un timeout LLM sur CPU : 3 × la durée

Que réessaie LangGraph par défaut ?

default_retry_on, lu dans le code de LangGraph 1.2

Exception	Réessayée ?
`ConnectionError`	✅
HTTP 5xx (httpx, requests)	✅
ValueError, TypeError, KeyError, RuntimeError	❌
OSError… y compris TimeoutError	❌
NodeTimeoutError (LangGraph)	✅ (hérite d'Exception)
toute autre exception	✅

💡 Le « Tips » du Formateur

Soyez explicite : retry_on=ConnectionError. Un argument invalide restera invalide ; une supervision lente le restera dans la seconde qui suit ; un timeout LLM sur CPU réessayé 3 fois = 3 × 2 minutes.

TimeoutPolicy : uniquement sur des nœuds async

```python
b.add_node("pinger", pinger_sync, timeout=TimeoutPolicy(run_timeout=1))
b.compile()
# ❌ ValueError: Node timeouts are only supported for async nodes …

async def pinger(state):
    return {"resultat": await RESEAU.ping(state["hote"])}

b.add_node("pinger", pinger, timeout=TimeoutPolicy(run_timeout=0.5))
await graphe.ainvoke({"hote": "srv-mail-01"})
# 💥 NodeTimeoutError: Node 'pinger' exceeded its run timeout of 0.500s
```

Paramètre	Rôle
`run_timeout`	durée maximale d'une tentative
`idle_timeout`	durée maximale sans progrès (streaming)

💡 Le « Tips » du Formateur

Réglez-le sur le matériel réel : 30 à 60 s par appel sur CPU, quelques secondes sur GPU.

🔎 TimeoutPolicy : borner la durée d'une tentative

Le délai s'applique à chaque tentative ; au-delà, NodeTimeoutError

```
📦 langgraph 1.2

from langgraph.types import TimeoutPolicy

b.add_node("pinger", pinger, timeout=TimeoutPolicy(run_timeout=0.5))
await graphe.ainvoke({"hote": "srv-mail-01"})
```

À quoi ça sert

Limite la durée d'UNE tentative d'un nœud. Au-delà, LangGraph annule la tâche asyncio et lève NodeTimeoutError, que la RetryPolicy peut réessayer et que l'error_handler peut rattraper. L'annulation repose sur asyncio : le nœud doit être async et le graphe lancé par ainvoke ; un nœud synchrone avec timeout est refusé dès compile().

↩️ Renvoie

Un objet de configuration. Au dépassement : NodeTimeoutError (langgraph.errors).

Paramètre	Dans le cours	Pourquoi · autres choix
`run_timeout`	`0.5, 1, 120.0 (jalon)`	plafond en secondes (ou timedelta), jamais prolongé ; à régler sur le matériel
`idle_timeout`	non utilisé	durée maximale SANS progrès, relancée à chaque signal (streaming de tokens)
`refresh_on`	défaut ("auto")	"auto" : progrès du graphe et heartbeats ; "heartbeat" : runtime.heartbeat() seulement
timeout=(add_node)	`TimeoutPolicy`	un float ou un timedelta = run_timeout seul
avec une RetryPolicy	Ex2.2, jalon J2	délai par tentative, pas pour l'ensemble des tentatives

⚠️ Pièges

- Nœud synchrone : ValueError à la compilation
- time.sleep ou calcul bloquant : le timeout ne part qu'après
- NodeTimeoutError réessayée par défaut : préciser retry_on

Le gestionnaire d'erreur, en code

```python
from langgraph.errors import NodeError

def repli(state, error: NodeError) -> Command:
    message = (f"supervision indisponible ({type(error.error).__name__} "
               f"dans « {error.node} ») : escalade N2")
    return Command(update={"resultat": message, "journal": ["repli activé"]},
                   goto="conclure")                  # sinon le graphe s'arrête

b.add_node("pinger", pinger,
           retry_policy=RetryPolicy(max_attempts=3, initial_interval=0.1,
                                    retry_on=ConnectionError),
           error_handler=repli)
b.add_node("conclure", conclure)   # START -> pinger -> conclure -> END
```

▶ **Sortie exacte (journal tronqué pour la slide)**

```
panne passagère  | 2 appel(s) | ["ping réussi à l'appel n°2", 'conclusion : srv-erp-02 : aucune …']
panne durable    | 3 appel(s) | ['repli activé', 'conclusion : supervision indisponible
                                (ConnectionError dans « pinger ») : escalade N2']
```

Et aussi

- set_node_defaults(…) : une politique pour tous les nœuds
- surcharge possible nœud par nœud

🔎 error_handler et NodeError : le repli

La fonction appelée quand un nœud a définitivement échoué

📦 langgraph 1.2

```python
from langgraph.errors import NodeError

def repli(state, error: NodeError) -> Command:
    return Command(update={"resultat": "…"}, goto="conclure")
b.add_node("pinger", pinger, retry_policy=…, error_handler=repli)
```

À quoi ça sert

error_handler désigne une fonction appelée quand un nœud échoue pour de bon : retries épuisés, timeout, exception non réessayée. Elle reçoit l'état et, via le paramètre error, le contexte de la panne. Elle décide de la suite avec une Command : réponse de repli, ToolMessage d'erreur, escalade. C'est le dernier filet avant que l'exception ne fasse tomber tout le graphe.

↩️ Renvoie

Comme un nœud : une Command (ou un dict). Le handler s'exécute après la dernière tentative, jamais à chaque échec.

Paramètre	Dans le cours	Pourquoi · autres choix
`error: NodeError`	paramètre du handler	le NOM error ET l'annotation NodeError sont requis pour l'injection
`error.node`	`"pinger"`	nom du nœud en échec : utile pour un handler partagé
`error.error`	`ConnectionError(...)`	l'exception d'origine : type(error.error).__name__ pour le message
retour Command(goto=)	`"conclure", "agent", END`	pour continuer ; un dict seul met à jour puis ARRÊTE le graphe
`set_node_defaults(error_handler=)`	§5	un handler pour tous les nœuds ; add_node le surcharge

⚠️ Pièges

- Handler sans goto : le graphe s'arrête après le repli
- def repli(state, panne: NodeError) : rien n'est injecté, TypeError
- Un handler qui lève : la run échoue, pas de handler du handler

🔎 add_node et set_node_defaults : les options de robustesse

En LangGraph 1.2, la robustesse se déclare au montage du graphe

📦 langgraph 1.2

```
b.add_node("tools", ToolNode(OUTILS),
           retry_policy=RetryPolicy(max_attempts=3),
           timeout=TimeoutPolicy(run_timeout=10.0),
           error_handler=outils_en_echec)
b.set_node_defaults(retry_policy=RetryPolicy(max_attempts=3))
```

À quoi ça sert

add_node accepte, en plus du nom et de la fonction, des options qui règlent le comportement du nœud sans toucher à son code. set_node_defaults fixe ces politiques pour tous les nœuds du graphe ; elles sont appliquées à compile(), et une valeur passée à add_node gagne toujours. Le jalon J2 règle ainsi agent, tools et qualifier, chacun avec son repli.

↩️ Renvoie

add_node et set_node_defaults renvoient le builder lui-même : on peut chaîner les appels.

Paramètre	Dans le cours	Pourquoi · autres choix
retry_policy	RetryPolicy	une politique, ou une liste : la 1re qui correspond
timeout	TimeoutPolicy	float, timedelta ou TimeoutPolicy ; nœuds async seulement
error_handler	outils_en_echec	fonction (state, error: NodeError) → Command
cache_policy	non utilisé	met en cache le résultat du nœud selon ses entrées
destinations	non utilisé	destinations d'un nœud à Command, pour le dessin seulement
defer	défaut (False)	True : exécuter le nœud à la fin du run
set_node_defaults	§5	retry et timeout valent aussi pour les handlers ; non hérité par les sous-graphes

⚠️ Pièges

- Timeout par défaut + un nœud synchrone : refus à la compilation
- Les défauts ne passent pas dans les sous-graphes

Le piège du ToolNode qui avale les erreurs

handle_tool_errors	Comportement	RetryPolicy du nœud tools
défaut	erreurs d'arguments renvoyées au modèle ; les autres REMONTENT	✅ fonctionne
`True`	TOUTE exception devient un ToolMessage d'erreur pour le modèle	❌ ne se déclenche jamais

Avec True…

c'est le LLM qui décide de réessayer, ou d'inventer un résultat. Pour une panne réseau, on préfère un retry déterministe.

Si les retries échouent

- un ToolMessage(status="error") PAR appel en attente
- puis Command(goto="agent")
- sinon : historique invalide pour le modèle

Le constater : handle_tool_errors=True contre le défaut

Un ToolNode s'exécute dans un graphe : ici, un graphe d'un seul nœud tools

```
from langgraph.prebuilt import ToolNode
from helpdesk_ops.reseau import ping_serveur

def graphe_outils(**options_toolnode):          # START -> tools -> END
    b = StateGraph(MessagesState)
    b.add_node("tools", ToolNode([ping_serveur], **options_toolnode))
    ...

# un AIMessage qui demande un outil, écrit à la main
appel = AIMessage("", tool_calls=[{"name": "ping_serveur",
                                   "args": {"hote": "srv-erp-02"}, "id": "appel-1"}])
RESEAU.configurer(echecs_avant_succes=99)       # supervision en panne

sortie = await graphe_outils(handle_tool_errors=True).ainvoke({"messages": [appel]})
print(sortie["messages"][-1].status, ":", sortie["messages"][-1].content[:60])
await graphe_outils().ainvoke({"messages": [appel]})   # défaut
```

▶ **Sortie exacte**

```
error : Error: ConnectionError('supervision injoignable (appel n°1)')
ConnectionError: supervision injoignable (appel n°2)      <- l'exception remonte
```

Lecture

- True : aucune exception, l'erreur part au modèle comme un résultat
- défaut : l'exception remonte, la RetryPolicy du nœud tools peut agir

🔎 ToolNode : le nœud qui exécute les outils

handle_tool_errors décide qui gère une panne : le modèle, ou la RetryPolicy du nœud

📦 langgraph-prebuilt 1.1

```python
from langgraph.prebuilt import ToolNode

b.add_node("tools", ToolNode([ping_serveur]))     # défaut
ToolNode([ping_serveur], handle_tool_errors=True)  # le piège
```

À quoi ça sert

Le nœud tout fait qui lit les tool_calls du dernier AIMessage, exécute les outils demandés et ajoute un ToolMessage par appel dans la clé messages. Il sert de nœud tools dans la boucle ReAct (jalons J1 et J2). handle_tool_errors décide de ce qui arrive quand un outil lève : renvoyer l'erreur au modèle comme un résultat, ou laisser l'exception remonter jusqu'à la RetryPolicy et l'error_handler du nœud.

↩️ Renvoie

Un nœud à passer à add_node ; à l'exécution il renvoie {"messages": [ToolMessage, …]}.

Paramètre	Dans le cours	Pourquoi · autres choix
`tools`	`[ping_serveur]`	fonctions @tool ou BaseTool
`handle_tool_errors`	défaut	défaut : erreurs d'ARGUMENTS au modèle, le reste remonte ; True : tout au modèle
autres valeurs	non utilisé	str, type(s) d'exception, fonction ⟶ str ; False : rien n'est capté
`name`	défaut ("tools")	nom du nœud
`messages_key`	défaut ("messages")	clé de l'état lue et écrite
`wrap_tool_call`	non utilisé	intercepter chaque appel (journal, contrôle)

⚠️ Pièges

- handle_tool_errors=True : la RetryPolicy du nœud ne voit jamais l'erreur
- Avec True, le LLM peut inventer un résultat
- Appelé seul, hors graphe : Missing required config key

Répondre à chaque appel d'outil en attente

Après un AIMessage à tool_calls, le modèle exige un ToolMessage par appel, sinon l'historique est invalide

```
from langchain_core.messages import ToolMessage

dernier = AIMessage("", tool_calls=[
    {"name": "ping_serveur",   "args": {"hote": "srv-erp-02"},  "id": "appel-1"},
    {"name": "statut_serveur", "args": {"nom": "srv-mail-01"},  "id": "appel-2"}])

reponses = [ToolMessage(content="refusé : maintenance en cours",
                        tool_call_id=a["id"], name=a["name"], status="error")
            for a in dernier.tool_calls]

for r in reponses:
    print(r.tool_call_id, r.name, r.status, r.content)
```

▶ **Sortie exacte**

```
appel-1 ping_serveur error refusé : maintenance en cours
appel-2 statut_serveur error refusé : maintenance en cours
```

Les 3 liens

- tool_call_id ← a["id"]
- name ← a["name"]
- status="error"

💡 Le « Tips » du Formateur

Dans un gestionnaire d'erreur, les appels en attente sont ceux du dernier message de l'état.

🔎 ToolMessage : la réponse à UN appel d'outil

Un tool_call sans ToolMessage = un historique que le serveur refuse

📦 langchain-core 1.6

```python
from langchain_core.messages import ToolMessage

ToolMessage(content="refusé : maintenance en cours",
            tool_call_id=a["id"], name=a["name"], status="error")
```

À quoi ça sert

Le message qui porte le résultat d'un appel d'outil. Le modèle exige qu'à chaque tool_call de son dernier AIMessage réponde un ToolMessage de même tool_call_id. ToolNode les crée d'habitude ; on les écrit à la main dans un repli (Ex2.2, jalon J2), quand l'outil n'a pas pu s'exécuter, pour garder un historique valide avant de revenir au nœud agent.

↩️ Renvoie

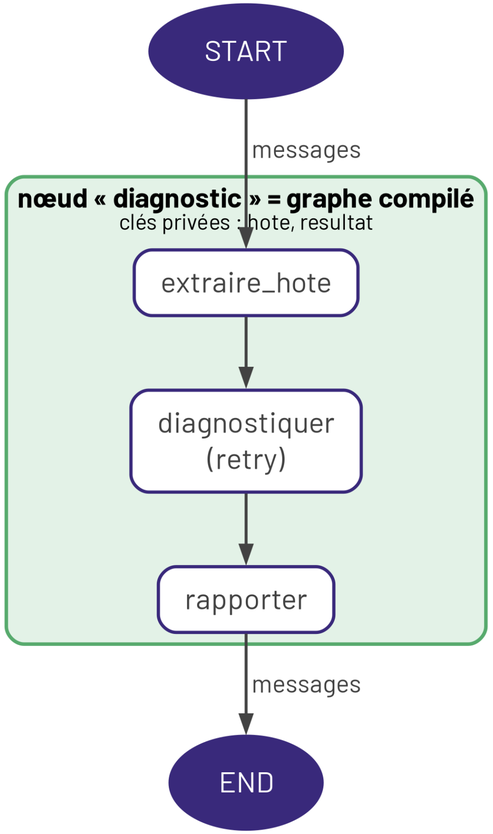
Un message à ajouter à l'état : Command(update={"messages": reponses}, goto="agent").

Paramètre	Dans le cours	Pourquoi · autres choix
`content`	texte du refus	ce que lira le modèle : dire clairement que l'outil a échoué
`tool_call_id`	`a["id"]`	OBLIGATOIRE : relie la réponse à l'appel
`name`	`a["name"]`	facultatif (défaut None), plus lisible dans les traces
`status`	`"error"`	défaut "success" ; "error" marque l'échec
`artifact`	non utilisé	données brutes gardées dans le message, non envoyées au modèle

⚠️ Pièges

- Oublier un appel quand il y en a plusieurs : historique invalide
- Un tool_call_id inventé : la réponse ne correspond à aucun appel
- Une erreur présentée comme un résultat : le modèle croit à un succès

Sous-graphes : composer sans tout mélanger




```python
class EtatDiagnostic(MessagesState):
    hote: str        # clés privées
    resultat: str

sous_graphe = sb.compile()            # extraire → diagnostiquer → rapporter

pb = StateGraph(MessagesState)
pb.add_node("diagnostic", sous_graphe)  # un graphe compilé = un nœud
```

- **Réutiliser :** le diagnostic sert à plusieurs agents
- **Tester :** chaque bloc séparément
- **Tracer :** étapes internes imbriquées dans Langfuse

🔎 Un graphe compilé comme nœud : le sous-graphe

add_node accepte un graphe compilé comme n'importe quelle fonction

📦 langgraph 1.2

```
sous_graphe = sb.compile()          # checkpointer=None : hérite
pb = StateGraph(MessagesState)
pb.add_node("diagnostic", sous_graphe)
```

À quoi ça sert

Un graphe compilé est un Runnable : add_node l'accepte tel quel. Le parent lui passe son état ; les clés communes (ici messages) circulent dans les deux sens, les clés propres au sous-graphe (hote, resultat) restent invisibles du parent. On teste chaque bloc séparément, on le réutilise, et Langfuse imbrique ses étapes. Chaque spécialiste multi-agent du J3 sera construit ainsi.

↩️ Renvoie

add_node renvoie le builder. À l'exécution, le nœud renvoie au parent les valeurs des clés communes.

Paramètre	Dans le cours	Pourquoi · autres choix
checkpointer (compile)	défaut (None)	None : hérite du checkpointer du parent ; False : aucun ; ne pas lui en donner un second
schéma d'état	`EtatDiagnostic(MessagesState)`	hériter du schéma parent garde les clés communes
subgraphs=True (get_state)	non utilisé	voir l'état interne d'un sous-graphe en pause
`graph=Command.PARENT`	non utilisé	depuis le sous-graphe, router dans le parent (handoffs, J3)
`set_node_defaults`	—	non hérité : régler la robustesse dans chaque sous-graphe

⚠️ Pièges

- Une clé privée utile au parent : la copier dans une clé commune
- Retry et timeout du parent : à redéclarer dans le sous-graphe

🧪 À vous : J2-04

14h00–15h30 · notebook J2_04_robustesse (cellules async : await directement)

- **§3** RetryPolicy : 2 échecs, puis succès au 3e appel
- **§4** TimeoutPolicy : refusée sur un nœud synchrone, NodeTimeoutError sur un nœud async
- **§5** error_handler + Command(goto) ; set_node_defaults
- **§6** le piège handle_tool_errors=True
- **§7** un sous-graphe de diagnostic comme nœud
- **Exercice 2.2 :** démo commentée, à refaire en autonomie

Slides à revoir

- RetryPolicy en code
- TimeoutPolicy · error_handler
- ToolMessage par appel

Réglez sur votre matériel

Timeout de l'agent : 30 à 60 s sur GPU, bien plus sur CPU.

🧰 Boîte à outils de l'exercice 2.2

Vous ne changez que les options passées aux nœuds agent et tools, et deux gestionnaires d'erreur

Étape	Syntaxe	Revoir
Options d'un nœud	`OPTIONS_TOOLS = {"retry_policy": …, "timeout": …, "error_handler": …}`	RetryPolicy en code
Retry	`RetryPolicy(max_attempts=3, initial_interval=0.2, retry_on=ConnectionError)`	RetryPolicy en code
Timeout	TimeoutPolicy(run_timeout=1)(nœud async : agent et ToolNode le sont)	TimeoutPolicy
Signature d'un repli	`def outils_en_echec(state, error: NodeError) -> Command:`	Gestionnaire d'erreur
Un ToolMessage par appel	`ToolMessage(content=…, tool_call_id=a["id"], name=a["name"], status="error")`	Répondre à chaque appel
Retour au modèle	`return Command(update={"messages": reponses}, goto="agent")`	Gestionnaire d'erreur
Repli de l'agent	`Command(update={"messages": [AIMessage("…")]}, goto=END)`	Gestionnaire d'erreur
Lancer	await essayer(robuste, scenario, f"ex22-{INITIALES}")(fourni)	énoncé, cellule 3

🧪 Exercice 2.2 : un outil réseau instable

exercices/enonces/Ex2_2_reseau_instable.ipynb · démo commentée (format 4 jours)

- **Constat :** 4 scénarios (stable, instable, panne, lente) sans protection
- **Retry :** 3 tentatives sur ConnectionError, sur le nœud tools
- **Timeout :** 1 s sur tools ; réglé pour votre machine sur agent
- **Replis :** ToolMessage d'erreur par appel + goto ; réponse polie si l'agent échoue
- **Preuve :** aucune exception, et les tentatives visibles dans Langfuse

